



Topic 例程 使用说明

V1.5.2

2026.05

文档版本

版本号	修订日期
1.4	2025 年 07 月 15 日
1.5	2025 年 07 月 23 日
1.5.1	2026 年 01 月 09 日
1.5.2	2026 年 05 月 13 日

目录

文档版本	I
1.在 windows 环境中使用 topic	3
2. topic 程序说明	8
附录	9

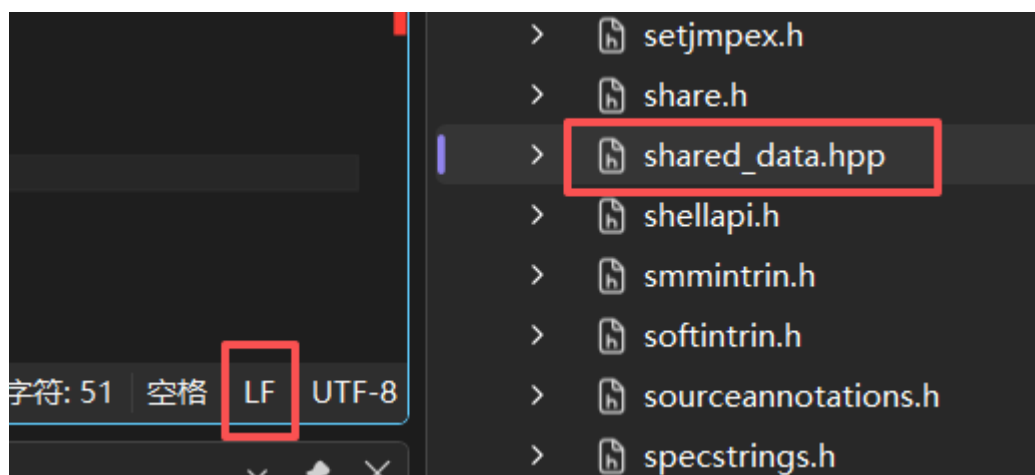
1.在 windows 环境中使用 topic

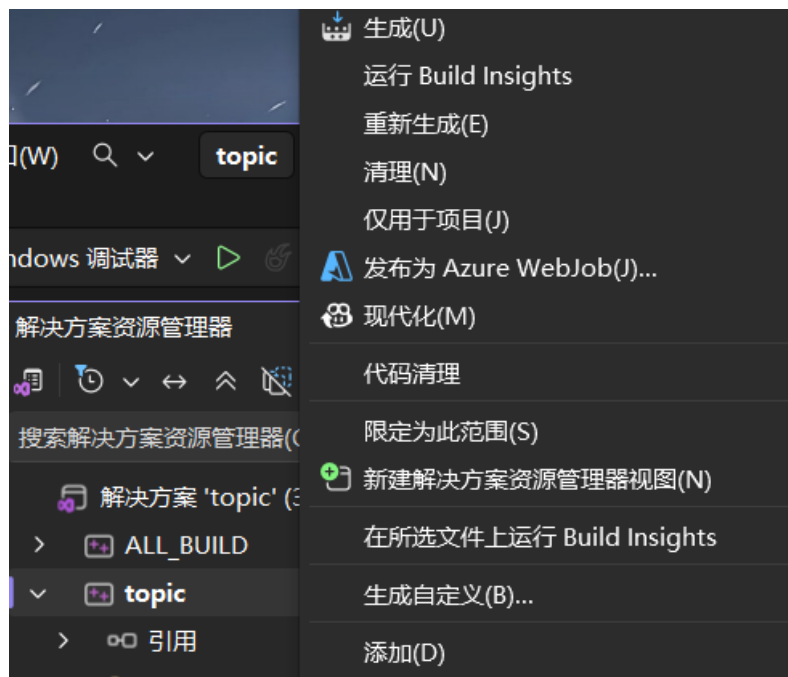
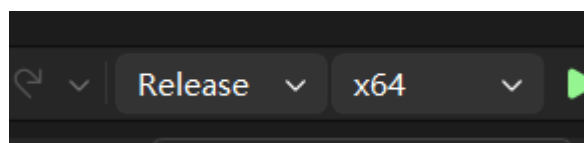
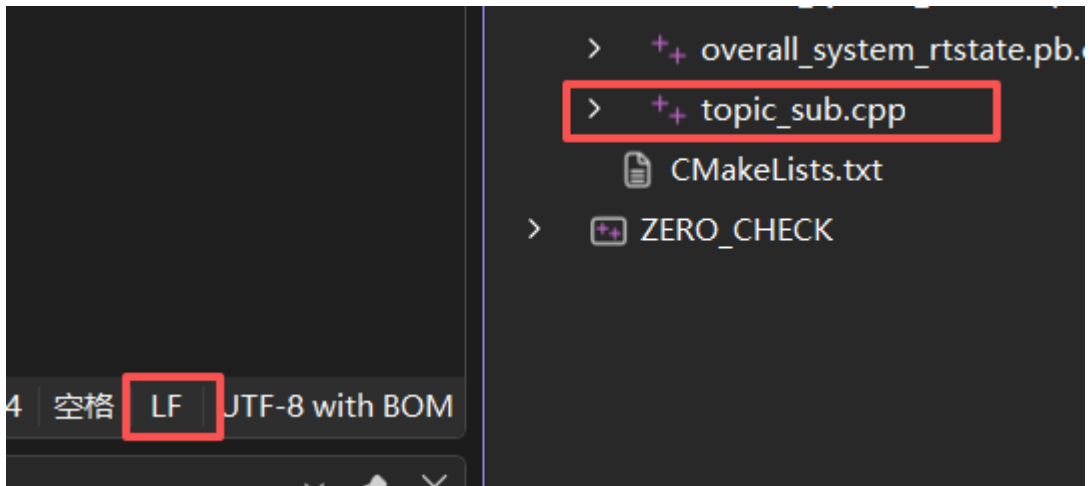
(1) 首先参考《Hello_MoveAbsJ_win_c++例程使用步骤》中的 CMake 操作，生成 .sln 解决方案文件

接着在 build 文件夹中打开 cmake 生成的 topic 工程

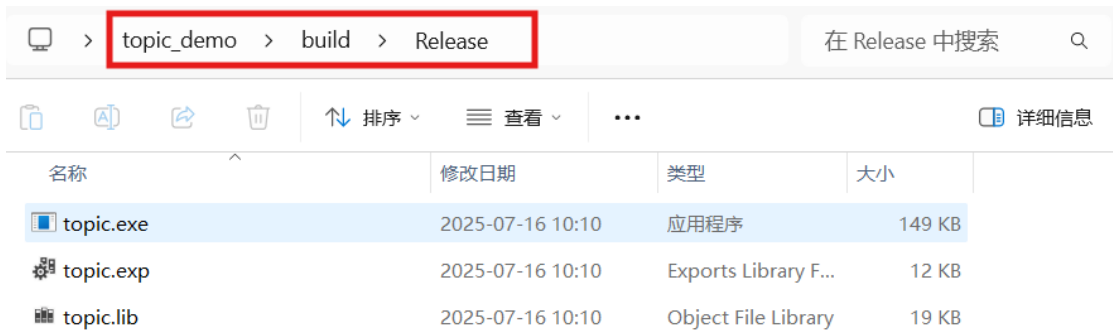
ALL_BUILD.dir	2026/5/19 11:12
CMakeFiles	2026/5/19 11:12
generated_proto	2026/5/19 11:12
topic.dir	2026/5/19 11:12
ZERO_CHECK.dir	2026/5/19 11:12
ALL_BUILD.vcxproj	2026/5/19 11:12
ALL_BUILD.vcxproj.filters	2026/5/19 11:12
cmake_install.cmake	2026/5/19 11:12
CMakeCache.txt	2026/5/19 11:12
topic.slnx	2026/5/19 11:12
topic.vcxproj	2026/5/19 11:12
topic.vcxproj.filters	2026/5/19 11:12
ZERO_CHECK.vcxproj	2026/5/19 11:12
ZERO_CHECK.vcxproj.filters	2026/5/19 11:12

(2) 在外部依赖项中 share_data.hpp 和 topic_sub.cpp 的行尾从 LF 改成 CRLF，选择 Release，选中 topic 右键点击设为启动项目并生成





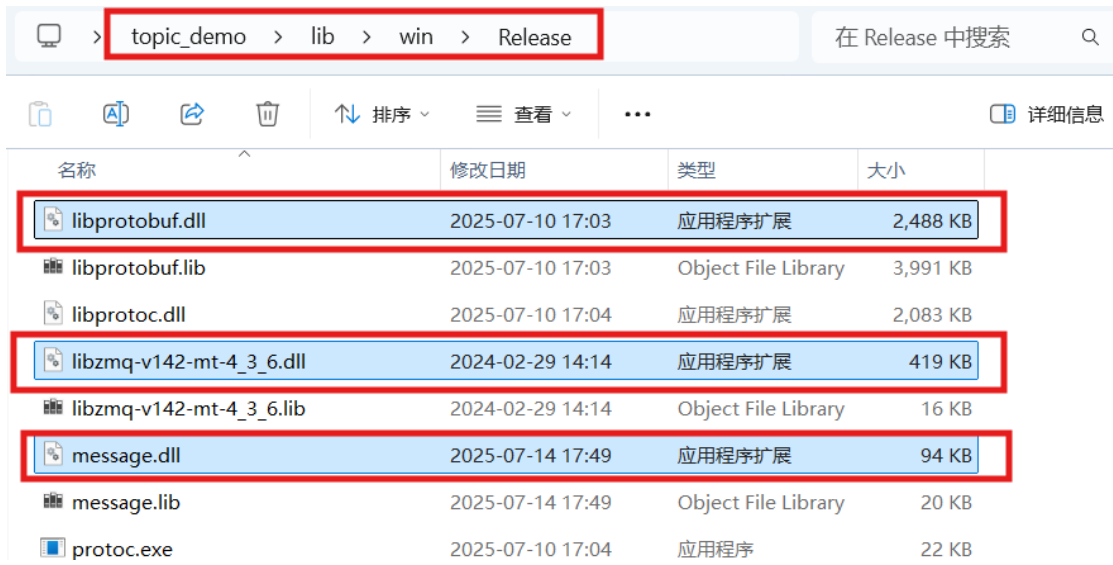
(3) 在 Release 文件夹中会生成对应的可执行文件



Windows File Explorer view of the 'Release' folder. The address bar shows the path: > topic_demo > build > Release. The search bar contains '在 Release 中搜索'. The file list shows three files:

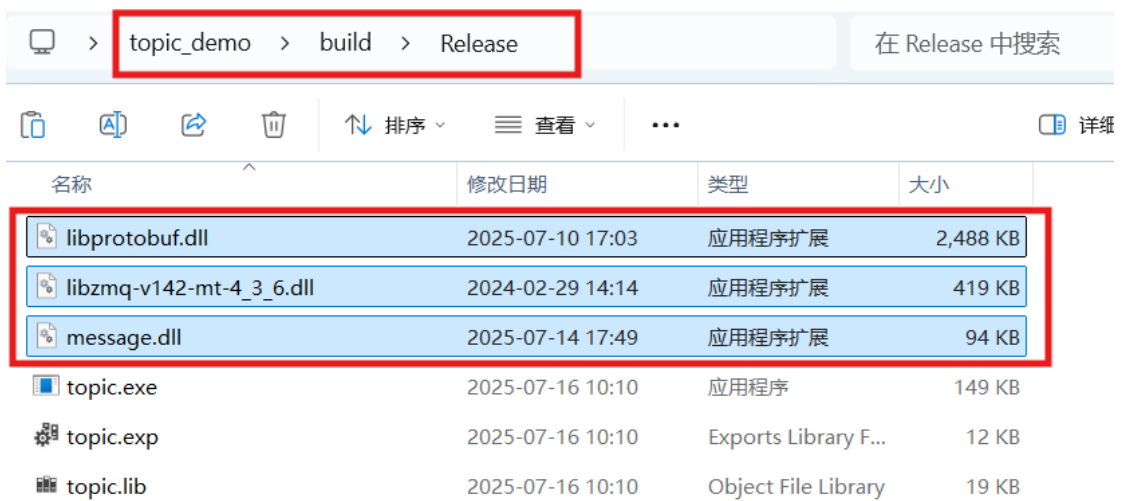
名称	修改日期	类型	大小
topic.exe	2025-07-16 10:10	应用程序	149 KB
topic.exp	2025-07-16 10:10	Exports Library F...	12 KB
topic.lib	2025-07-16 10:10	Object File Library	19 KB

(4) 然后将 topic_demo\lib\win\Release 中的 libprotobuf.dll、libzmq-v142-mt-4_3_6.dll 及 message.dll 复制粘贴到 topic_demo\build\Release 目录下



Windows File Explorer view of the 'Release' folder in 'topic_demo\lib\win'. The address bar shows the path: > topic_demo > lib > win > Release. The search bar contains '在 Release 中搜索'. The file list shows several files, with three highlighted by red boxes:

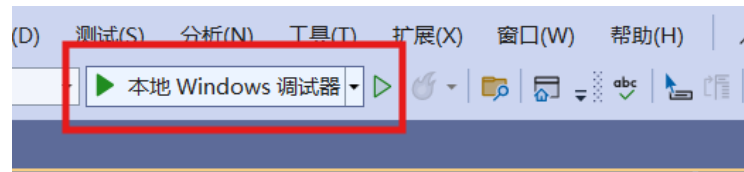
名称	修改日期	类型	大小
libprotobuf.dll	2025-07-10 17:03	应用程序扩展	2,488 KB
libprotobuf.lib	2025-07-10 17:03	Object File Library	3,991 KB
libprotoc.dll	2025-07-10 17:04	应用程序扩展	2,083 KB
libzmq-v142-mt-4_3_6.dll	2024-02-29 14:14	应用程序扩展	419 KB
libzmq-v142-mt-4_3_6.lib	2024-02-29 14:14	Object File Library	16 KB
message.dll	2025-07-14 17:49	应用程序扩展	94 KB
message.lib	2025-07-14 17:49	Object File Library	20 KB
protoc.exe	2025-07-10 17:04	应用程序	22 KB



Windows File Explorer view of the 'Release' folder in 'topic_demo\build' after copying files. The address bar shows the path: > topic_demo > build > Release. The search bar contains '在 Release 中搜索'. The file list shows the same files as before, plus the three copied DLL files highlighted by a red box:

名称	修改日期	类型	大小
libprotobuf.dll	2025-07-10 17:03	应用程序扩展	2,488 KB
libzmq-v142-mt-4_3_6.dll	2024-02-29 14:14	应用程序扩展	419 KB
message.dll	2025-07-14 17:49	应用程序扩展	94 KB
topic.exe	2025-07-16 10:10	应用程序	149 KB
topic.exp	2025-07-16 10:10	Exports Library F...	12 KB
topic.lib	2025-07-16 10:10	Object File Library	19 KB

(5) 最后在工程中点击调试，出现窗口并持续收到产品发来的状态信息



```
NodeOptions
  name: {test1}
  pub url: {}
  sub url: {tcp://127.0.0.1:19091}
  thread poll size: {1}
  pub queue length: {50}
  sub queue length: {1000}
init
on_monitor_started
zmq_version 4.3.6
[SUB] connect: {tcp://127.0.0.1:19091}
sub_monitor_thread_ started.
Creating subscription for topic: system_rtstate
LoopRecv started...
NodeOptions
  name: {test2}
  pub url: {}
  sub url: {tcp://127.0.0.1:19091}
  thread poll size: {1}
  pub queue length: {50}
  sub queue length: {1000}
init
on_monitor_started
zmq_version 4.3.6
[SUB] connect: {tcp://127.0.0.1:19091}
sub_monitor_thread_ started.
Creating subscription for topic: system_nrtstateLoopRecv started...
```

(6) sub.url 填写对应发布端 IP，127.0.0.1 为本地 IP，端口号固定为 19091

在使用时需将 IP 改为 192.168.11.11

根据需求在 display 函数下方打印需要查看的参数

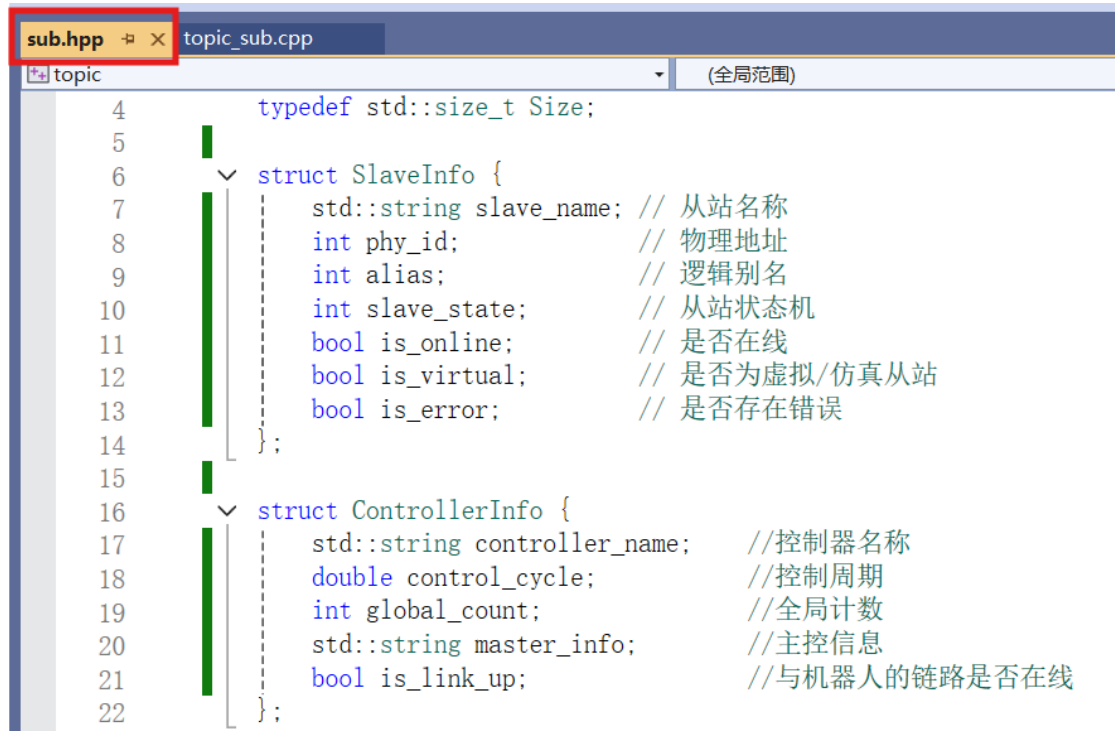
```
NodeOptions system_rtstate_options;
system_rtstate_options.node_name = "test1";
system_rtstate_options.sub_url = "tcp://127.0.0.1:19091";
Node system_rtstate_node(system_rtstate_options);
system_rtstate_node.Start();

auto subrt = system_rtstate_node.CreateSubscription<system_rtstate::SystemRtState>("system_rtstate", [](const system_rts
  SystemStateData rtparm;
  display rt(tt, rtparm);
  std::cout << "rthead:" << rtparm.header_timestamp << " " << rtparm.header_frame_id << std::endl;
  std::cout << "system_running_state:" << rtparm.system_running_state << std::endl;
});
```

```
rthead:21835 152
system_running_state:1
```

```
rthead:21835 152
system_running_state:1
```

(7) topic_sub.cpp 中引用的 sub.hpp 文件中，定义的结构体包含了可打印的参数及其含义，可根据需求在 main 中的 display 后添加打印查看



```

4         typedef std::size_t Size;
5
6     struct SlaveInfo {
7         std::string slave_name; // 从站名称
8         int phy_id;           // 物理地址
9         int alias;            // 逻辑别名
10        int slave_state;       // 从站状态机
11        bool is_online;        // 是否在线
12        bool is_virtual;       // 是否为虚拟/仿真从站
13        bool is_error;         // 是否存在错误
14    };
15
16    struct ControllerInfo {
17        std::string controller_name; // 控制器名称
18        double control_cycle;        // 控制周期
19        int global_count;             // 全局计数
20        std::string master_info;     // 主控信息
21        bool is_link_up;             // 与机器人的链路是否在线
22    };
    
```

2. topic 程序说明

程序分为几个部分：配置 ip，订阅远程地址，实时数据处理获取，打印硬件模型资源等信息。

程序通过轮询全局状态接口获取数据。它的主循环中以高频率（500ms）调用 `getRt()` 接口实时获取 `system_rtstate`，用于监控机器人瞬时状态；同时在同一循环中调用 `getNrt()` 接口获取 `system_nrtstate`，用于读取系统配置与静态参数。RT 数据更新速度快，反映当前时刻的状态，而 NRT 数据更新相对较慢，主要用于系统配置管理。

程序中已经将所以接口信息全部打印，可以根据需求自行取用。

```
// 配置&创建节点 配置远程地址 192.168.2.10:19091 并创建topic节点
// 分别订阅 system_rtstate 及 system_nrtstate 主题
NodeOptions system_topic_options;
system_topic_options.node_name = "test1";
system_topic_options.sub_url = "tcp://192.168.11.11:19091";
Node system_topic_node(system_topic_options);
system_topic_node.Start();

===== RT =====
auto subrt = system_topic_node.CreateSubscription<system_rtstate::SystemRtState>("system_rtstate", [](const system_rtstate::SystemRtState& tt) [...]);

===== NRT =====
auto subnrt = system_topic_node.CreateSubscription<system_nrtstate::SystemNrtState>("system_nrtstate", [](const system_nrtstate::SystemNrtState& tt) [...]);

===== RT Basic =====
auto subrt_basic = system_topic_node.CreateSubscription<system_rtstate::SystemRtState>("system_rtstate", [](const system_rtstate::SystemRtState& tt) [...]);

while (true) {
    std::this_thread::sleep_for(std::chrono::microseconds(20));
}
return 0;
```

```
auto subrt = system_rtstate_node.CreateSubscription<system_rtstate::SystemRtState>("system_rtstate", [](const system_rtstate::SystemRtState& tt) {
    SystemStateData rtparm;
    display_rt(tt, rtparm);

    //*****
    std::cout << "1.rtheadr: " << rtparm.header_timestamp << " " << rtparm.header_frame_id << std::endl;
    std::cout << "2.system_running_state: " << rtparm.system_running_state << std::endl;

    std::cout << "3.Controller Info:" << std::endl;
    std::cout << " 3.1.controller_name: " << rtparm.controller.controller_name << std::endl;
    std::cout << " 3.2.control_cycle: " << rtparm.controller.control_cycle << std::endl;
    std::cout << " 3.3.global_count: " << rtparm.controller.global_count << std::endl;
    std::cout << " 3.4.master_info: " << rtparm.controller.master_info << std::endl;
    std::cout << " 3.5.is_link_up: " << rtparm.controller.is_link_up << std::endl;

    std::cout << "4.Models Info:" << std::endl;
    for (size_t i = 0; i < rtparm.models.size(); ++i) {
        std::cout << " 4. " << i << " Model " << i << " " << std::endl;
        std::cout << " 4a.model_name: " << rtparm.models[i].model_name << std::endl;
        std::cout << " 4b.model_type: " << rtparm.models[i].model_type << std::endl;

        std::cout << " 5.Joints Info:" << std::endl;
        for (size_t j = 0; j < rtparm.models_joints.size(); ++j) {
```

注：std::count 的打印方式没有线程保护，直接观察打印信息会存在顺序错乱问题属于正常现象，现有打印方式仅提供各接口以及调用方法，可根据需求进行选取调用。

附录

法律声明

1. 版权声明

© 2025 贞实科技版权所有。未经书面授权，禁止以任何形式复制、传播本文件内容。

2. 免责条款

使用者应自行承担因以下情况导致的损失风险：

- 未按指引操作造成的系统配置错误
- 擅自修改动态库文件引发的程序崩溃（如 message.dll 等库非授权替换）
- 开发环境版本不匹配导致的编译失败（如 CMake 2.8 与 VS2022 兼容问题）

3. 数据隐私条款（Data Privacy）

产品运行过程中产生的日志文件，仅包含设备状态信息，不会收集用户个人信息。